The Open University

Mathematics and Computing: Level 2
M256 Software development with Java

M256

Glossary

WEB 91942 7
1.1

A

abstract class A class which cannot be instantiated, i.e. one which has no instances of its own but only instances of its child classes. An abstract class defines properties which are common to its child classes. It should not be confused with a class representing abstract entities.

abstract entity An entity that has an existence, but does not correspond to a tangible entity. For example, the film Citizen Kane is an abstract entity, while the DVD which records the film is a tangible entity.

Abstract Windowing Toolkit (AWT) Java classes, defined in the package `java.awt`, including classes implementing events and layout managers.

abstraction A description that focuses on the essential features of a problem and ignores other details.

acceptance testing Testing which aims to convince the client that the system works as specified.

acceptance tests A series of tests devised to demonstrate that a completed system can meet all the behavioural requirements set out as use cases in the requirements document.

access modifier One of the keywords `public`, `protected` and `private`, specifying the extent to which class members are accessible from outside the class. In the absence of one of these keywords the default package access applies.

action button See button.

activation rectangle An element in a sequence diagram that represents a period during which a particular object is active.

adaptive method A method of software development which embraces change by building space for it into the schedule.

affordance The manipulations that something (e.g. a widget) appears to allow; that is, the actions that it appears should be carried out by the user to manipulate that thing.

agile method An approach to software development typically involving a small team working on an ambitious project, operating outside normal company policies and to a tight schedule.

alternative scenario A (valid) scenario for which the use case requirements specify an alternative course of action.

analysis (in software development) In this context, analysis involves analysing the specified requirements and expressing, in computing terms, what the software system should do.

analyst A generic term for a person who takes on the role (or a combination of the roles) of a business analyst, systems analyst or requirements analyst.

animate agent An independently acting cause of change in the human environment.

anthropomorphism The attribution of human personality to what is not human (in the case of this course, to software entities).

application A program that performs a specific function directly for the user and, in effect, turns the computer into a specialised computer, such as a word processor or web browser.

application programming interface (**API**) In M256 this term refers to the predefined Java packages. More generally, the services offered by a component may be referred to as the component's API.

architecture A term used in different ways in software engineering, the common theme being concentration on the overall organisation of a system: viewing the system in some abstract, high-level way, ignoring smaller details.

assertion A JUnit test statement.

association An association represents the links that may exist between the objects of two classes. These links have a common meaning, and represent a particular relationship (connection) between these objects that is significant for the system domain.

attribute An attribute is a property of an object. All objects of a class have the same attributes.

B

back end See storage tier.

behaviour (of a software system) The set of tasks the system performs.

behaviour (of an object) An object's set of responsibilities.

behavioural requirement A specific task that the system being developed must be able to do.

black box testing Testing in which a method is tested against its specification, without any examination of code.

block tag A part of a doc comment which contains information about particular elements of a method, such as return values, parameters, etc.

boundary A point at which the software's specified behaviour changes.

buckets Storage compartments used when storing elements in a collection, such as a set.

bug A defect in software.

business analyst A person who understands the business systems within an organisation.

business area The real-world context within which the system is required to operate.

business domain tier In a tiered architecture, a tier corresponding to the core system. This contains the parts of the system specific to a business area, which process data and generate information.

business system A business system is a combination of people and software working together to meet a particular set of business objectives.

button A widget typically used to enable the user to initiate a single action.

C

call (a method) See invoke.

candidate class A class that may be needed to model a category of real-world entities from the system domain.

cascading A design where, to get an object `r` to perform a task, an object `p` sends a message to an object `q` asking it to ask `r` to perform its task, and `q` sends a message to `r` asking it to perform its task.

CASE (computer-aided software engineering) tool A software tool used to help in some aspect of software development.

check box One of a set of widgets (there may be only one) enabling choice (including none) from a set of options which are not mutually exclusive.

child class A class that is a specialisation of some other class, known as its parent class; a subclass.

class (in software) A template that serves to describe all instances (objects) of that class. It defines the type of data held by the objects and the objects' operations.

class (in the conceptual model) A class models a category of real-world entities from the system domain.

class description A formal description of a class within a conceptual model or a structural model.

class diagram Class diagrams are used to show structural aspects of the system domain and of a system at various stages in its development. As a model of the system domain, a class diagram depicts conceptual classes and associations connecting them. As a model of the system, a class diagram depicts the software classes that are involved in the system and the associations connecting them.

class member An element of a class definition, such as an instance variable, method or constructor.

client This term has two main meanings in the context of software development: (i) the object in a collaboration which requests a service: (ii) the person(s) commissioning the software.

client–server architecture A logical architecture consisting of two tiers: a client and a server.

code-based testing tool A testing tool that automatically analyses code and produces test cases.

coding tool A CASE tool that aids writing or running code.

cohesion A measure of how strongly related and focused the responsibilities of a component are. High cohesion is considered desirable.

collaboration One object requesting a service from another object.

collaborator A participant in a collaboration.

command button See button.

command-line interface A user interface which requires the user to enter the names of commands and the names of the files (or other items of interest) to be manipulated.

component On this course, unless indicated to the contrary, taken to mean an object-oriented software component.

component-based approach An approach to software development based on the philosophy of reusing existing components, and of structuring the software into interacting components which may be reused.

component-based software Software created using a component-based approach.

composition Reusing a class through an instance of the class being part of the state of an object of a new class.

computer architecture A view of the different computer elements that a system uses – hardware elements such as different processors and so on.

conceptual model The model which represents the significant entities in the system domain and the connections between them as a collection of classes and their attributes, associations and invariants. The conceptual model of the system domain is taken to be the initial structural model for the software system to be developed. It consists of a class diagram, class descriptions and invariants.

concrete class A class that, unlike an abstract class, may have instances that are not instances of any of its child classes.

concurrency control Regulating access to shared data to prevent problems associated with concurrent access.

concurrent access (to a system) Simultaneous use of a system by two or more users or clients.

constraint (on the system) A limitation on the system or its development.

constraint (on the system domain) A limitation on the entities in the system domain, their properties and relationships.

container A Swing component which contains other visual components. Every Swing GUI must have a top-level container in which the rest of the GUI components are nested.

coordinating class The class of the coordinating object.

coordinating message A message from the user interface to the coordinating object to initiate the core system behaviour required for a use case, or part of a use case.

coordinating method The method invoked on receipt of a coordinating message.

coordinating object The core system object through which the user interface accesses the services of the core system.

copy constructor A constructor that returns a copy of its argument, i.e. a different object of the same class with the same state.

core object An object contained within the core system.

core system The part of the system that is distinct from the user interface and that usually contains objects corresponding to real-world entities.

coupling A measure of the extent to which a component is dependent on other components. Low coupling is considered desirable.

D

data hiding Protecting an object's implementation details by restricting access to them.

decompiler A tool that takes compiled Java files and derives the source code.

defensive copy (of an object) A distinct object with the same state as the original.

dependency There is a dependency, or usage, between two components when elements in one component rely on, or in some way use, elements in the other.

deployment How the various components that go to make up a distributed system are physically arranged, i.e. what physical computer or computers each part of the software runs on, and how these computers communicate.

deployment diagram A diagram illustrating how a system's components are deployed.

derived association An association all of whose links can be derived using other elements of the model.

derived attribute An attribute all of whose values can be derived using other elements of the model.

design Deciding how the system will meet the specified requirements.

design for all See universal access.

design for reuse Producing components as part of developing new software, or specifically as marketable commodities.

design guideline (user interface design) A piece of guidance for the design of user interfaces, which typically is detailed and relevant to only specific kinds of system or specific contexts.

design heuristic (user interface design) A piece of relatively concrete advice for designing a user interface: a rule of thumb that may not always be applicable, but is frequently useful. (Also referred to as a *usability heuristic* or simply a *heuristic*.)

design pattern (software design) A tried and tested outline design applicable across a range of software systems, describing a recurring theme in elegant and respected designs. Typically, the description takes the form of a 'solution' which can be applied to a 'problem' which may arise in different contexts. Design patterns are commonly classified as *structural* (concerned with the system's structure), *creational* (concerned with how objects are created), or *behavioural* (concerned with interactions within the system).

design principle (software design) A guideline for which objects should be allocated which kinds of task, and how the objects should communicate.

design principle (user interface design) A piece of guidance for the design of user interfaces, which typically is more abstract than a design guideline, and which should apply more or less universally to any kind of system in any context.

design rationale (user interface design) Documentation of a designer's suggested improvement to a user interface, in the context of a particular design heuristic.

design tool A CASE tool that aids some aspect of design.

design with reuse Using existing components in creating a new piece of software.

designer A developer whose role is to work on the design stages of a project.

detailed design and implementation Deciding which existing classes can be reused and what programming constructs are appropriate as well as writing the actual code.

developing a conceptual model Analysing the requirements to determine the classes and connections between them that appropriately model the key concepts in the real-world area the system is being written for.

developing a user interface Designing the user interface and determining how it will communicate with the core system.

developing dynamic models Designing and comparing models of the interactions among objects which will achieve the tasks required of the system.

dialogue box A particular type of window which gives the user information, and may also allow the user to enter information and to select from a set of options.

direct manipulation A style of interaction based on the metaphor of physical manipulation.

distributed system A system involving more than one computer connected by a network.

doc form A form of Java comment marked by `/**` at the beginning and `*/` at the end.

dynamic model A model illustrating actions and interactions within the system when the system is in operation. In this course, a dynamic model consists of a walk-through and a sequence diagram, in the context of a particular scenario.

Dynamic Systems Development Method (DSDM) A significant form of RAD devised by the not-for-profit DSDM Consortium.

E

effectiveness (in the context of a user interface) How well the user interface enables the user to get the right thing done.

efficiency A measure of how well a system carries out its tasks.

efficiency (in the context of a user interface) How much of a resource has to be expended for a user to get a job done via the user interface.

encapsulation The packaging together in an object of data and the operations that can be applied to that data.

engineering components The tangible artefacts designed and constructed by engineers.

enumerated type (enum) A user-defined Java type that can take on only one of a finite set of values specified by the programmer when the type is defined.

event (in a GUI) Something that happens when a user interacts with a GUI; typically initiated by a mouse operation, although the keyboard also generates events. In Java an object is created to represent each event that occurs. Code must be associated with an event in order that the system can react appropriately to it.

event (in the system domain) Any significant circumstance, episode, interaction, happening or incident, e.g. deliveries, registrations, bookings, enrolments.

event handler method A method in which event handling code is written.

event handling code Code that specifies the action to be taken in response to a particular event.

extraordinary HCI Human–computer interaction in situations where the user has special needs.

eXtreme Programming (XP) An example of an agile method.

F

façade class A class added to a component, as described by the Façade pattern, to provide a primary means of interaction with the component. In M256, such a class is termed the *coordinating class*.

façade object An instance of a façade class. In M256, such an object is termed the *coordinating object*.

Façade pattern An example of a structural design pattern.

feedback Information about the results of the user's action.

first code iteration The first stage in implementing a core system. In M256, this stage implements the basic structure of each class, i.e. its attributes, link references, constructors, attribute getter methods and `toString()` method.

fixture The state of the relevant parts of the system before a particular test.

forking A design where, to get an object `r` to perform a task, an object `p` sends a message to an object `q` asking it for `r`; `q` returns `r` to `p`, and `p` then sends a message to `r` asking it to perform the task.

forwarding method A method in which the corresponding message is forwarded to an object being used by composition.

front end See user interface tier.

functionality (of a widget) The types of tasks the widget enables a user to perform, and the types of communication between user and system it mediates.

G

generalisation A relationship between classes that captures the similarities and differences between two related classes. Saying that class `A` is a generalisation of class `B` means that the common (similar) structure and behaviour is defined by class `A`, and the distinctive (different) structure and behaviour is defined by class `B`. It means that an instance of class `B` is substitutable for an instance of class `A`.

getter method A method enabling clients to discover the value of an instance variable.

graphical user interface (GUI or GUI interface) A user interface that typically combines elements such as windows, buttons and menus: visual elements which can be manipulated by the user to interact with the system.

H

hashcode A value returned by a `hashCode()` method, used by Java in hashing.

hashing A technique that speeds up searching a collection.

helper method A private method that exists simply to assist the work of another method in the same class.

heuristic See design heuristic.

heuristic evaluation An assessment of a user interface with reference to particular design heuristics.

human–computer interaction (HCI) The study of how computers and people work together. The term HCI is used to refer both to the scientific study of how humans and computers interact, and (the focus in this course) to the practical business of designing user interfaces to enable such interactions.

I

icon A small representational picture used in a GUI; for example, to label a button.

identifier A label that is used to refer to an object in a system.

immutable object An object whose state cannot be changed.

implementation Translating a design into program code.

implementation details (of a class) How the class is coded, including the code for its methods and constructors, and the declaration of its instance variables.

implementation model A model specifying the actual classes that will be used to implement the system. It contains a structural model and dynamic models. Its structural model is the final result of the evolution of the initial structural model.

incremental prototyping Successively modifying and augmenting a prototype until it meets all the requirements.

incremental software development An approach to software development where the requirements of the software are partitioned and partitions developed separately.

information expert An object which most readily has to hand the information needed to carry out a particular task.

information flow The input from the user and the output from the system.

information system A system which involves calculating and providing information for the user.

inherit Take on the properties (attributes and associations) of a parent class.

in-house standards A company's specified standards to which its software must adhere.

initial structural model The initial structural model is the conceptual model viewed as the first model of the structure of the proposed software system. In other words, the conceptual and initial structural models 'look' the same but have different perspectives: one models the system domain; the other is taken as the first model of the software system itself.

initial widget table A table setting out the information flow and the widgets to be used in the initial prototype screen for a particular use case.

instance Refers to a particular object of a class or a particular link of an association.

integrated development environment (IDE*)* A software tool which facilitates many of the tasks associated with writing and running programs in a specific language.

integration testing Testing that problems do not arise from the interaction of a system's components.

integrity (of a package) The consistency of the states of the objects in the package with any invariants on the objects.

integrity (of an object) The validity of an object's state, that is, the consistency of the values of its instance variables with any invariants on the object.

interaction design The design of any aspects of a system that affect the quality of a user's interaction.

interactive product Any kind of computer-based device that requires users to engage with it.

interface type An interface declared as the type of a variable or method answer.

invalid scenario A scenario where a pre-condition is broken.

invariant Any condition on the objects of a conceptual model, their attribute values and links that must hold for the requirements to be satisfied and the consistency of the model to be guaranteed.

invoke (a method) To execute a method's code.

iteration One cycle through the phases involved in an iterative method.

iterative method An adaptive method of software development in which phases are repeated iteratively in a systematic manner.

J

JAR file A form of zip file used to distribute compiled Java code, documentation and other information required for software to be used.

Java application A program run directly by the Java Virtual Machine.

Javadoc A software tool which extracts doc comments from a Java program and from them provides a description of elements of the program, such as methods, output in the form of an HTML file.

JUnit A software tool that assists with testing Java classes.

L

label Information, usually textual, about a widget.

large-scale project A project in which the amount and complexity of detail involved cannot readily be handled by one or two people, so that the project is best carried out by a team of people, with different individuals working on different aspects of the project.

layer See tier.

layout manager A Java object that controls how Swing components can be laid out in a container.

learnt affordance A view of affordance which stresses that the way a user perceives how something should be manipulated is not a property of the thing alone, but a property of the combination of the thing and the user's background, skill, capabilities and training.

lifeline An element in a sequence diagram that represents the time during which an object exists.

link A link exists between two objects when there is a connection between the entities they represent that is significant for the system domain. A link is an instance of an association.

list A widget that allows the user to select one (or, rarely, more) relevant pieces of information to provide to the system.

list box A widget which combines a text box and a list.

listener An instance of the class in which a handler method is implemented.

lock A means of providing uninterrupted access to an object.

logical architecture A view of a system's software which is concerned with the organisation of the software classes into larger components, and which concentrates on how the overall system is constructed from major components, how the components are connected and how they work together.

M

maintainability The extent to which software is easily maintained.

maintenance The phase of software development associated with keeping the system working to the satisfaction of its users.

mental model Specific existing knowledge, often of the physical world, which users are typically expected to possess, and upon which a user interface metaphor is based.

menu A widget which groups together several items from which the user may select. Pull-down, pop-up and contextual menus are examples of different types of menu.

menu bar A widget that takes the form of a bar, appearing typically along the top of an application window, and which provides a way of hierarchically grouping commands. The words along the menu bar represent major groupings of commands, arranged in menus.

menu interface A user interface which presents the user with a sequence of sets of options from each of which the user selects, typically by pressing a button or typing in a number.

message A request by one object for another object to provide a service.

method (in software development) See software development method.

modelling language A specification of how models should be constructed so that their meaning is unambiguous.

model–view separation principle A software design principle which states not only that the business domain (the 'model') and the user interface (the 'view') should be separate entities, but that the business domain should not depend on the user interface.

multiple releases (of software) Prototypes handed over to the client at various stages of the software's development.

multiplicity The multiplicity of an association with respect to one of the classes involved defines how many objects of that class may be linked with a single object of the other class involved in the association.

multi-user system A system which several users may access via different computers.

mutable object An object whose state can be changed.

N

NASA TLX (Task Load Index) tool A tool that measures a user's impression of their workload when carrying out some task, based on a multidimensional rating procedure that calculates an overall workload score based on a weighted average of ratings on six subscales: frustration, mental demands, physical demands, temporal demands, own performance and effort.

navigate To access an object or objects from another object by using a link or links of an association.

network architecture A view of how the different computers involved in a system are connected.

noun phrase A noun phrase is a phrase (a sequence of words like 'number of copies') that functions as a noun and which can be used anywhere that a noun could be used.

O

object (in software) A collection of data and a set of operations that can be applied to the data.

object (in the conceptual model) An object models a real-world entity from the system domain.

object diagram An illustration of objects and the links between them.

Object Management Group (OMG) A consortium of computing companies which sets standards across the software industry, including the UML standards.

object-oriented software component One or more classes grouped together in some way to form a piece of software that has a well-defined purpose and that can be combined with other pieces of software to construct a larger system.

one-way association An association that is navigated in only one direction.

one-way link A link of a one-way association.

open source software Software developed using open source software development.

open source software development An approach to software development whose key feature is that source code is made publicly available on a website, free for anybody to copy, modify and use (subject to licence agreements).

orchestrating object See coordinating object.

organisational unit Any part of an organisational structure to which people or things in the system domain belong.

P

package A Java mechanism for grouping classes together to form an object-oriented software component.

package diagram A diagram illustrating a system's components and their dependencies.

parent class A class that is a generalisation of one or more other classes known as its child classes; a superclass.

persistence The ability of a system to save state from one execution to the next.

phase A stage of software development.

polymorphism The capability for objects of different classes to respond to the same message in a manner appropriate for each object.

portability A measure of how easily a system can run on different platforms.

post-condition What is achieved by invoking a method, if its pre-conditions are satisfied.

pre-condition A restriction on the circumstances under which a method may be invoked.

predictive method A method of software development which is largely based on the waterfall method and which therefore benefits from simplicity of planning, and predictability.

privacy leak A situation where it may be possible for a client to access and improperly manipulate the state of an object.

program documentation Comments and explanations designed to assist someone in understanding the implementation of software, what it does and how to use it.

programmer A developer whose role is to implement the code.

project documentation A written description of the activities, decisions and outcomes of a project's phases.

project failure A situation where a project fails to deliver the client's requirements in some way (e.g. in terms of cost, development time, or functionality).

project manager A person who plans and oversees the running of a software development project.

protocol (of a class) The specifications of the methods defined in a class, together with the specifications of inherited methods, which define the corresponding messages that its objects will respond to.

protocol (of a component) The combined protocols of the component's classes, defining how clients can interact with the component.

prototype An early working version of a system or part of it.

Publish–Subscribe pattern An example of a behavioural design pattern.

R

radio button One of a set of widgets for enabling choice between mutually exclusive options, precisely one of which must be selected.

rapid application development (RAD) A software development method based on timeboxing.

Rational Unified Process (RUP) A description of a set of iterative software development methods having in common a set of 'best practice' elements. The RUP was devised by the software development experts who were also involved in developing the UML.

reasoning by class An innate form of human reasoning, derived from a form of reasoning about animals, which can be described as follows.

If we observe that just one animal of a particular unfamiliar type has a certain kind of behaviour (e.g. bites, flies, lays eggs), then it is a fairly good assumption that any other animal of that type will do the same.

reasoning by inheritance An innate form of human reasoning, derived from a form of reasoning about animals, which can be described as follows.

If we are given the single piece of information that one type of animal (animal A) is a special kind of another animal (animal B) then any time we encounter an animal of type A, we will know that it can do everything that animal Bs do, and probably some extra things.

recursive association An association where the links connect objects belonging to the same class.

refactoring The process of amending a system's structure, while retaining its behaviour, in order to improve its quality.

regression testing Rerunning previous tests to check that a change or addition made to one aspect of a system has not damaged another, previously working aspect.

Remote Method Invocation (RMI) A Java mechanism that allows a program to send messages to a Java object which is in a different JVM and running on a different computer.

requirements What is required of the system.

requirements analysis The process of analysing and distilling the information from requirements elicitation into a clear and coherent description of the system and specification of its behaviour.

requirements analyst A person who is an expert in eliciting and analysing the requirements of a new software system.

requirements creep A situation where the software developers come under pressure to deliver in excess of what was originally agreed.

requirements document The specification of the requirements of the system produced after negotiation between the client and the analyst.

requirements elicitation The act of finding out the requirements of a system by consulting with the stakeholders.

requirements specification Eliciting and analysing what the client wants in order to produce a detailed and complete specification of the tasks required (i.e. the required functionality) of the system.

responsibility (of a component) The services that the component offers.

responsibility (of an object) The tasks, or services, carried out by the object on receipt of appropriate messages.

reusability A measure of the extent to which existing artefacts have been used during the development of the software in question, and the extent to which the software itself offers artefacts for reuse.

review A point within an iterative software development method where developers and clients can take changes into account.

role The particular responsibility or job that a person (thing) may perform in the system domain, e.g. employee, student, lecturer, driver.

S

safety-critical system A system operating in an environment in which safety is particularly crucial.

scenario An example (instance) of a use case involving particular example objects.

scope The breadth of services that the system is expected to provide.

screen See window.

scroll pane A widget that displays scrollbars as and when they are required. If there are too many items, a vertical scroll bar will automatically appear. If an item is too wide, a horizontal scrollbar will automatically appear.

sequence diagram An illustration of objects collaborating to carry out a particular task.

serialisable (object) A serialisable object is one whose class implements the `Serializable` interface and can therefore be represented in a way that enables it to be stored in a file (i.e. persist) or sent over a network.

server The object in a collaboration that provides a service.

setter method A method enabling clients to change the value of an instance variable in a controlled way.

singleton An object that is the only instance of its class, as described by the Singleton pattern.

Singleton pattern An example of a creational design pattern.

Sketchpad The first computer system to make a substantial contribution to the discipline of HCI, implemented by Ivan Sutherland in 1962 as part of his PhD thesis.

slider A widget that acts as a device to input or output a number or other value, by moving a bar.

socio-technical system A system comprising both human and technical elements.

software See software system.

software component A piece of software with a well-defined purpose that can be combined with other pieces of software to construct a larger system. On this course, unless the context indicates otherwise, the term is taken to mean object-oriented software component.

software developer An umbrella title, referring to someone who takes on one or more of a range of jobs within software development.

software development A planned, phased process, involving modelling different aspects of the software as well as implementing, testing and maintaining it.

software development method A particular set of development phases applied in a particular order.

software engineering A term used to refer to a wide range of concerns connected with carrying out systematic software development.

software model An illustration or description of the software, or of part of it, which emphasises certain aspects and omits others.

software system A program which is large in the sense that it carries out a number of tasks, some of which may be complex.

software tester A developer whose role is to test the software as it is being developed.

specialisation Saying that a class `B` is a specialisation of class `A` means that class `A` is a generalisation of class `B`.

specification (of a method) A description of the purpose and usage of a method, including its pre-conditions and post-conditions.

specification (of an engineering component) A description of the engineering component's interface, requirements and services.

specification (of an object-oriented software component) A description of how to interact with the component, and of its requirements and services.

stakeholder Anyone who has an interest in a system including the client, the future users, analysts and developers.

state (of a system) The objects, their attribute values and the links between them, which constitute the system at a particular time.

state (of an object) The values of the object's attributes.

static model An illustration of the state of the system, or part of it, at a particular time.

stereotype A label providing additional information about some component of a UML model. Stereotypes are enclosed in double chevrons, e.g. `<<abstract>>`.

storage tier In a tiered architecture, a tier which provides a persistent data-storage mechanism, such as a database. Also known as the back end of the system.

storyboard A diagrammatic representation of a user interface, which may show the navigation through screens.

strict UML diagram A diagram that adheres strictly to the UML standard.

structural model A representation of the system in terms of classes and the relationships between them.

structured English A description of code which has some of the structure of code but without the syntax of a programming language.

subclass See child class.

substitutable, **substitutability** Objects of a class `B` are substitutable for objects of a class `A` if anywhere that an instance of class `A` is required, an instance of class `B` can be used instead. For class `A` to be a generalisation of class `B`, instances of `B` must be substitutable for instances of `A`.

superclass See parent class.

Swing Java classes, defined in the package `java.swing`, used for programming GUIs.

system See software system.

system code The source code which, when run, generates the system.

system domain The real-world context specific to the software system being developed.

system testing Testing the complete system against its specification once all the components are integrated, including testing aspects of the system such as performance, security, system start up, etc.

systems analyst A developer who is a technical expert in software systems and whose role is to analyse the feasibility of a proposed system and how it will impact on the client's business practices.

T

tabbed screen One of a set of screens, each with a tab at the top. To move from one screen to another, the tab of the desired screen is clicked on. Sometimes colloquially known as a tabbed pane.

tangible entity A physical 'thing' in the system domain such as an aeroplane, vehicle, reactor or person, i.e. something that can be seen or touched.

target language The language in which the planned software is to be implemented.

technical writer A developer whose role is to develop user documentation.

test In M256, the activity of checking that a specific aspect of the code is consistent with the implementation model, and hence that it correctly represents the designs.

test case A specification of the data to be used in a particular test, and the expected result.

test class A class in which are implemented the test methods for another class.

test driver tool A testing tool that executes the software being tested with specified inputs.

test method A method which tests another method.

test runner A feature of JUnit which enables the automatic execution of test methods and the reporting of results.

test suite A grouping of test classes.

testability A measure of how easily a system can be tested.

test-driven development An approach to coding and testing in which tests are written based on the specification of the aspect of the system under test, before the code is written.

testing The activities that take place at each phase of development to ensure that the phases of development are consistent and complete with respect each other, and also consistent and complete with respect to the requirements.

testing tool A CASE tool that aids some aspect of testing.

text area A widget which is like a text field but which can accommodate multiple lines of text.

text box See text field.

text field A widget providing an area into which the user can type, or which can be used to display information, or both. A text field accommodates just a single line of text.

think-aloud A simple usability testing technique which involves the user speaking aloud what they are thinking when using the user interface to perform tasks.

three-layered architecture See three-tiered architecture.

three-tiered architecture A logical architecture consisting of three tiers: a user interface tier, a business domain tier and a storage tier. Also known as a three-layered architecture.

tier An element in a system's logical architecture. Also known as a layer. A tier can be considered as a component with distinct responsibilities such that:

- a lower tier has lower-level responsibilities;
- higher tiers have responsibilities that are increasingly specific to the business area;
- higher tiers may depend on lower tiers but not vice versa.

tiered architecture A logical architecture which is structured as a series of tiers.

timeboxing The practice of partitioning the timescale of a software development project into timeboxes, each corresponding to an iterative cycle and in each of which requirements are prioritised. Timeboxing is a key feature of RAD.

top-level container A component which is the overall container for the components in a GUI.

two-way association An association that is navigated in both directions.

two-way link A link of a two-way association.

U

UML (Unified Modeling Language) A modelling language based on diagrams.

UML standard The currently accepted specification of what is valid UML and how it should be used.

UML-type diagram A diagram that varies in some small way from the strict UML standard.

unit testing Testing individual units of software in the system: in an object-oriented approach, units may be considered to be methods, or classes.

universal access An approach to development whose key idea is that effort should be expended to find simple ways in which small changes to a system could make it more widely accessible (also referred to as *design for all*).

unnamed package The default Java package for a user-defined class.

usability Refers to how easily and quickly a product enables users to achieve their goals, how quickly users learn to use the product's interface and what the users' attitude is towards the interface. The ISO 9241 (Part 11) definition of usability is:

> The extent to which a product can be used by specified users to achieve specified goals [tasks] with effectiveness, efficiency, and satisfaction in a specified context of use.

usability heuristic See design heuristic.

usability testing Assessing the usability of an interface.

usage See dependency.

use case A concise description of a specific task that a user of the system must be able to perform.

use case-driven An approach to implementation in which each use case is tackled one by one: having coded and tested the methods involved in one use case, another use case is tackled.

user interface The features of a system the user can use to communicate with or control the system, and those which the system can use to communicate with, or influence the user. A user interface has three aspects: hardware; visual elements; and interaction design.

user interface designer A software developer whose particular responsibility is the user interface.

user-centred approach An approach to developing software which focuses on what users find easy and what they find difficult.

user-centred design A design culture or design process where users' needs, wants, expectations and limitations are given serious, detailed attention at all stages.

user-defined type A type defined by the user, augmenting those provided by the Java API.

user interface metaphor A metaphor applied in the design of a user interface.

user interface tier In a tiered architecture, a tier giving a 'view' of the information provided by the system, using windows, spreadsheets, forms etc. Also known as the front end of the system.

user needs analysis The process of finding out as much as possible about the people who will be using the system.

user needs analysis document A record of the results of user needs analysis, containing information about users, tasks and situations, and giving user interface designers a concrete basis on which to work.

user satisfaction (in the context of a user interface) How satisfied the user is in interacting with the system via the user interface.

utility class A class which is not specific to the particular system under development, but which has potentially wider applicability.

V

valid scenario A scenario where no pre-condition is broken.

validation Checking that all aspects of the system and its project documentation are consistent with the customer's requirements. In other words, has the right system been built?

value (of an attribute) Specific information that an object holds in respect of an attribute.

verification Checking that all aspects of the system and its project documentation are consistent and complete with respect to each other. In other words, has the system been built right?

virtual reality system A system which gives the user a sense of three-dimensional direct experience or physical presence via cues that may be visual, aural or haptic, and which uses computer-generated 'objects' that are manipulated like their real-world equivalents by picking up, turning around, throwing etc.

visibility The extent to which the purpose of something (e.g. a widget) is apparent, i.e. how clear it is what the thing is for.

visual component A Java implementation of a widget.

W

walk-through A list of individual steps within the core system which carry out a use case in the context of particular scenario.

waterfall method A traditional and idealised view of developing software by strictly following a sequence of phases.

white box testing Testing which involves examination of code.

widget A visual element in a GUI such as a button, a list etc.

window A window- or container-like widget, typically allowing a view onto some aspect of a system or application.

working scenario Code that sets up a typical set of objects and implements the design for a use case in a very basic way as a 'proof of concept'.

X

Xerox Star A landmark computer system in HCI history, which laid the foundations of modern GUI interfaces and was associated with seminal user interface design ideas.